

MYSQL TRIGGERS

- **First Trigger Example – AFTER INSERT**

Create an employees table:

```
CREATE TABLE employees(  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    salary DECIMAL(10,2)  
);
```


Query 1 x

SQL Addition

Limit to 1000 rows

1 • `SELECT * FROM employees;`

Result Grid

emp_id	emp_name	dept_id	salary
NULL	NULL	NULL	NULL

employees 1 x

Apply Revert Context Help

Output

Action Output

#	Time	Action	Message
1	12:27:01	CREATE DATABASE trigger_demo	Error Code: 1007. Can't create databa
2	12:27:13	CREATE DATABASE trigger_demo	Error Code: 1007. Can't create databa
3	12:28:33	USE trigger_demo	0 row(s) affected
4	12:31:32	CREATE TABLE employees(emp_id INT PRIMA...	0 row(s) affected
5	12:32:19	SELECT * FROM employees LIMIT 0, 1000	0 row(s) returned

Create an audit table:

```
CREATE TABLE employee_audit(
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,
```

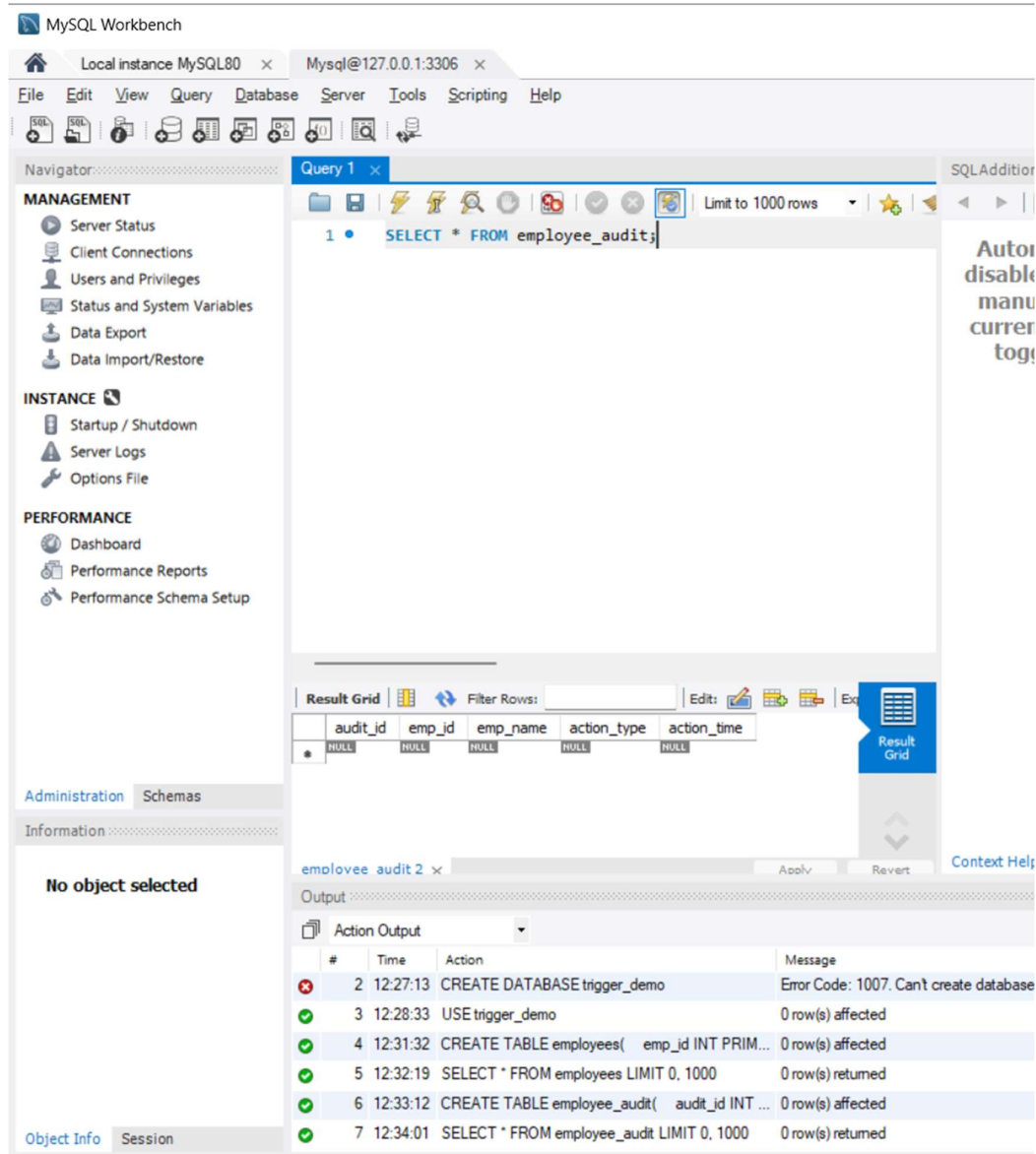
```
emp_name VARCHAR(50),  
action_type VARCHAR(20),  
action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

The screenshot shows a SQL IDE window titled "Query 1". The query editor contains the following SQL code:

```
1 CREATE TABLE employee_audit(  
2     audit_id INT AUTO_INCREMENT PRIMARY KEY,  
3     emp_id INT,  
4     emp_name VARCHAR(50),  
5     action_type VARCHAR(20),  
6     action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
7 );
```

Below the query editor is the "Output" pane, which is set to "Action Output". It displays a list of executed actions with their status, time, and messages.

#	Time	Action	Message
❌ 1	12:27:01	CREATE DATABASE trigger_demo	Error Code: 1007. Can't create databa
❌ 2	12:27:13	CREATE DATABASE trigger_demo	Error Code: 1007. Can't create databa
✅ 3	12:28:33	USE trigger_demo	0 row(s) affected
✅ 4	12:31:32	CREATE TABLE employees(emp_id INT PRIMA...	0 row(s) affected
✅ 5	12:32:19	SELECT * FROM employees LIMIT 0, 1000	0 row(s) returned
✅ 6	12:33:12	CREATE TABLE employee_audit(audit_id INT A...	0 row(s) affected



Now create the trigger:

DELIMITER \$\$

CREATE TRIGGER after_employee_insert

AFTER INSERT ON employees

FOR EACH ROW

BEGIN

INSERT INTO employee_audit

(emp_id, emp_name, action_type)

VALUES

(NEW.emp_id, NEW.emp_name, 'INSERT');

END \$\$

DELIMITER ;

The screenshot displays a SQL IDE interface. The top pane, titled 'Query 1', contains the following SQL code:

```
1 DELIMITER $$
2
3 CREATE TRIGGER after_employee_insert
4 AFTER INSERT ON employees
5 FOR EACH ROW
6 BEGIN
7     INSERT INTO employee_audit
8     (emp_id, emp_name, action_type)
9     VALUES
10    (NEW.emp_id, NEW.emp_name, 'INSERT');
11 END $$
12
13 DELIMITER ;
```

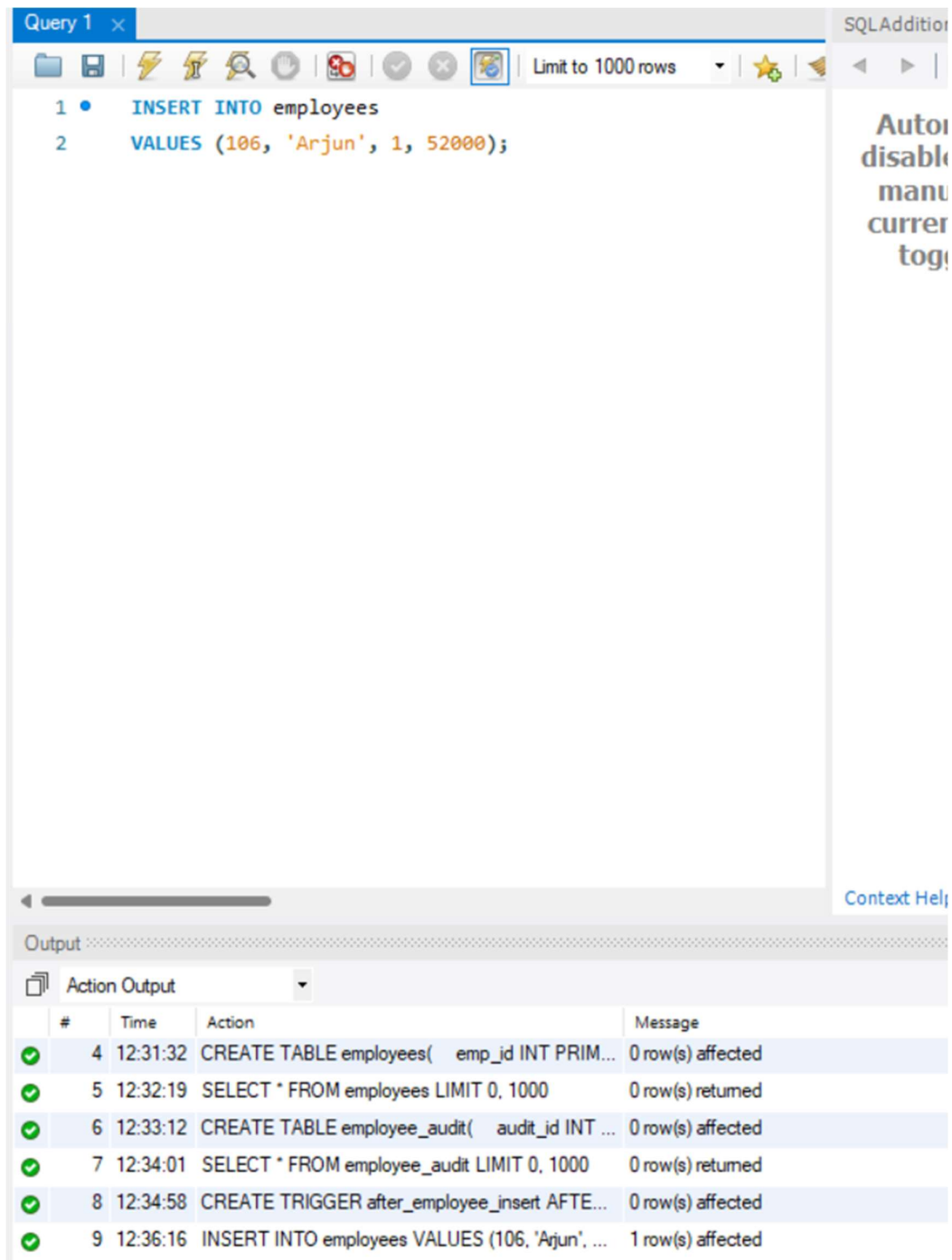
The bottom pane, titled 'Output', shows the 'Action Output' table with the following data:

#	Time	Action	Message
✓ 3	12:28:33	USE trigger_demo	0 row(s) affected
✓ 4	12:31:32	CREATE TABLE employees(emp_id INT PRIM...	0 row(s) affected
✓ 5	12:32:19	SELECT * FROM employees LIMIT 0, 1000	0 row(s) returned
✓ 6	12:33:12	CREATE TABLE employee_audit(audit_id INT ...	0 row(s) affected
✓ 7	12:34:01	SELECT * FROM employee_audit LIMIT 0, 1000	0 row(s) returned
✓ 8	12:34:58	CREATE TRIGGER after_employee_insert AFTE...	0 row(s) affected

Now insert an employee:

INSERT INTO employees

VALUES (106, 'Arjun', 1, 52000);



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The query editor contains two lines of SQL code:

```
1 • INSERT INTO employees
2   VALUES (106, 'Arjun', 1, 52000);
```

Below the query editor is the 'Output' pane, which is currently set to 'Action Output'. It displays a table of execution results:

#	Time	Action	Message
✓ 4	12:31:32	CREATE TABLE employees(emp_id INT PRIM...	0 row(s) affected
✓ 5	12:32:19	SELECT * FROM employees LIMIT 0, 1000	0 row(s) returned
✓ 6	12:33:12	CREATE TABLE employee_audit(audit_id INT ...	0 row(s) affected
✓ 7	12:34:01	SELECT * FROM employee_audit LIMIT 0, 1000	0 row(s) returned
✓ 8	12:34:58	CREATE TRIGGER after_employee_insert AFTE...	0 row(s) affected
✓ 9	12:36:16	INSERT INTO employees VALUES (106, 'Arjun', ...	1 row(s) affected

On the right side of the IDE, there is a sidebar with the text 'Auto disable manu curre togg' and a 'Context Help' link at the bottom.

➤ You **didn't call the trigger**. It automatically executes.

```
SELECT * FROM employee_audit;
```

Output:

```
106  Arjun  INSERT
```


Query 1 x

SQLAdditio

Limit to 1000 rows

1 • `SELECT * FROM employee_audit;`

Auto
disabl
man/
curre
tog

Result Grid

	audit_id	emp_id	emp_name	action_type	action_time
▶	1	106	Arjun	INSERT	2026-08-17 12:36:16
*	NULL	NULL	NULL	NULL	NULL

Result Grid

employee audit 3 x

Apply Revert Context He

Output

Action Output

#	Time	Action	Message
✓ 5	12:32:19	SELECT * FROM employees LIMIT 0, 1000	0 row(s) returned
✓ 6	12:33:12	CREATE TABLE employee_audit(audit_id INT ...	0 row(s) affected
✓ 7	12:34:01	SELECT * FROM employee_audit LIMIT 0, 1000	0 row(s) returned
✓ 8	12:34:58	CREATE TRIGGER after_employee_insert AFTE...	0 row(s) affected
✓ 9	12:36:16	INSERT INTO employees VALUES (106, 'Arjun', ...	1 row(s) affected
✓ 10	13:38:33	SELECT * FROM employee_audit LIMIT 0, 1000	1 row(s) returned

● Second Trigger Example – AFTER DELETE

Suppose Employee deleted:

DELETE FROM employees

WHERE emp_id = 106;

We can automatically record that deletion.

DELIMITER \$\$

CREATE TRIGGER after_employee_delete

AFTER DELETE ON employees

FOR EACH ROW

BEGIN

INSERT INTO employee_audit

(emp_id, emp_name, action_type)

VALUES

(OLD.emp_id, OLD.emp_name, 'DELETE');

END \$\$

DELIMITER ;

Query 1 x SQL Addition

Limit to 1000 rows

1 • `SELECT * FROM employee_audit;`

Result Grid

	audit_id	emp_id	emp_name	action_type	action_time
▶	1	106	Arjun	INSERT	2026-08-17 12:36:16
	2	106	Arjun	DELETE	2026-08-17 13:43:32
*	NULL	NULL	NULL	NULL	NULL

Result Grid

employee audit 4 x Apply Revert Context Help

Output

Action Output

#	Time	Action	Message
✓ 8	12:34:58	CREATE TRIGGER after_employee_insert AFTE...	0 row(s) affected
✓ 9	12:36:16	INSERT INTO employees VALUES (106, 'Arjun', ...	1 row(s) affected
✓ 10	13:38:33	SELECT * FROM employee_audit LIMIT 0, 1000	1 row(s) returned
✓ 11	13:41:25	CREATE TRIGGER after_employee_delete AFTE...	0 row(s) affected
✓ 12	13:43:32	DELETE FROM employees WHERE emp_id = 106	1 row(s) affected
✓ 13	13:43:59	SELECT * FROM employee_audit LIMIT 0, 1000	2 row(s) returned

● Third Trigger Example--AFTER UPDATE Trigger:

UPDATE employees

SET salary = 60000

WHERE emp_id = 101;

We can record the update.

Query 1 x SQLAddition

Limit to 1000 rows

```
1 INSERT INTO employees
2 VALUES (101, 'Rahul', 1, 50000);
```

Auton
disabl
manu
curre
toge

Context Help

Output

Action Output

#	Time	Action	Message
✓ 9	12:36:16	INSERT INTO employees VALUES (106, 'Arjun', ...	1 row(s) affected
✓ 10	13:38:33	SELECT * FROM employee_audit LIMIT 0, 1000	1 row(s) returned
✓ 11	13:41:25	CREATE TRIGGER after_employee_delete AFTE...	0 row(s) affected
✓ 12	13:43:32	DELETE FROM employees WHERE emp_id = 106	1 row(s) affected
✓ 13	13:43:59	SELECT * FROM employee_audit LIMIT 0, 1000	2 row(s) returned
✓ 14	13:45:27	INSERT INTO employees VALUES (101, 'Rahul', ...	1 row(s) affected

Query 1 x

SQL Addition

Limit to 1000 rows

1 • `SELECT * FROM employees;`

Result Grid

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	50000.00
*	NULL	NULL	NULL	NULL

employees 5 x

Apply Revert Context Help

Output

Action Output

#	Time	Action	Message
✓ 10	13:38:33	SELECT * FROM employee_audit LIMIT 0, 1000	1 row(s) returned
✓ 11	13:41:25	CREATE TRIGGER after_employee_delete AFTE...	0 row(s) affected
✓ 12	13:43:32	DELETE FROM employees WHERE emp_id = 106	1 row(s) affected
✓ 13	13:43:59	SELECT * FROM employee_audit LIMIT 0, 1000	2 row(s) returned
✓ 14	13:45:27	INSERT INTO employees VALUES (101, 'Rahul', ...	1 row(s) affected
✓ 15	13:46:03	SELECT * FROM employees LIMIT 0, 1000	1 row(s) returned

DELIMITER \$\$

CREATE TRIGGER after_employee_update

AFTER UPDATE ON employees

FOR EACH ROW

BEGIN

INSERT INTO employee_audit

(emp_id, emp_name, action_type)

VALUES

(NEW.emp_id, NEW.emp_name, 'UPDATE');

END \$\$

DELIMITER ;

Query 1 x

SQLAdditio

Limit to 1000 rows

```
1 DELIMITER $$
2
3 • CREATE TRIGGER after_employee_update
4 AFTER UPDATE ON employees
5 FOR EACH ROW
6 BEGIN
7     INSERT INTO employee_audit
8     (emp_id, emp_name, action_type)
9     VALUES
10    (NEW.emp_id, NEW.emp_name, 'UPDATE');
11 END $$
12
13 DELIMITER ;
```

Auto
disabl
man
curre
tog

Context Hel

Output				
Action Output				
#	Time	Action	Message	
✓ 11	13:41:25	CREATE TRIGGER after_employee_delete AFTE...	0 row(s) affected	
✓ 12	13:43:32	DELETE FROM employees WHERE emp_id = 106	1 row(s) affected	
✓ 13	13:43:59	SELECT * FROM employee_audit LIMIT 0, 1000	2 row(s) returned	
✓ 14	13:45:27	INSERT INTO employees VALUES (101, 'Rahul', ...	1 row(s) affected	
✓ 15	13:46:03	SELECT * FROM employees LIMIT 0, 1000	1 row(s) returned	
✓ 16	13:47:12	CREATE TRIGGER after_employee_update AFT...	0 row(s) affected	

Query 1 x SQLAddit

Limit to 1000 rows

1 • `SELECT * FROM employee_audit;`

Aut
disab
mar
curro
to

Result Grid

	audit_id	emp_id	emp_name	action_type	action_time
▶	1	106	Arjun	INSERT	2026-08-17 12:36:16
	2	106	Arjun	DELETE	2026-08-17 13:43:32
	3	101	Rahul	INSERT	2026-08-17 13:45:27
	4	101	Rahul	UPDATE	2026-08-17 13:47:57
*	NULL	NULL	NULL	NULL	NULL

Result Grid

employee audit 7 x Apply Revert Context H

Output

Action Output

#	Time	Action	Message
✓ 14	13:45:27	INSERT INTO employees VALUES (101, 'Rahul', ...	1 row(s) affected
✓ 15	13:46:03	SELECT * FROM employees LIMIT 0, 1000	1 row(s) returned
✓ 16	13:47:12	CREATE TRIGGER after_employee_update AFT...	0 row(s) affected
✓ 17	13:47:57	UPDATE employees SET salary = 60000 WHER...	1 row(s) affected Rows matched: 1 C
✓ 18	13:48:22	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned
✓ 19	13:49:09	SELECT * FROM employee_audit LIMIT 0, 1000	4 row(s) returned

● **Fourth Trigger Example--BEFORE INSERT Trigger:**

Employee salary cannot be less than ₹20,000.

```
DELIMITER $$

CREATE TRIGGER check_salary

BEFORE INSERT ON employees

FOR EACH ROW

BEGIN

    IF NEW.salary < 20000 THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Salary cannot be less than 20000';

    END IF;

END $$

DELIMITER ;
```

Query 1

SQLAddition

Limit to 1000 rows

```

1  DELIMITER $$
2
3  CREATE TRIGGER check_salary
4  BEFORE INSERT ON employees
5  FOR EACH ROW
6  BEGIN
7      IF NEW.salary < 20000 THEN
8          SIGNAL SQLSTATE '45000'
9          SET MESSAGE_TEXT = 'Salary cannot be less than 20000';
10     END IF;
11 END $$
12
13 DELIMITER ;

```

Auton

disabl

manu

curre

toge

Context Help

Output

Action Output

#	Time	Action	Message
✓ 15	13:46:03	SELECT * FROM employees LIMIT 0, 1000	1 row(s) returned
✓ 16	13:47:12	CREATE TRIGGER after_employee_update AFT...	0 row(s) affected
✓ 17	13:47:57	UPDATE employees SET salary = 60000 WHER...	1 row(s) affected Rows matched: 1 Cha
✓ 18	13:48:22	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned
✓ 19	13:49:09	SELECT * FROM employee_audit LIMIT 0, 1000	4 row(s) returned
✓ 20	13:50:16	CREATE TRIGGER check_salary BEFORE INSE...	0 row(s) affected

Now, run this Query

INSERT INTO employees

VALUES (107, 'Ravi', 1, 15000);

Query 1 x SQLAddition

Limit to 1000 rows

```
1 • INSERT INTO employees
2 VALUES (107, 'Ravi', 1, 15000);
```

Autodisable
manu
curren
toge

Context Help

Output

Action Output

#	Time	Action	Message
✓ 16	13:47:12	CREATE TRIGGER after_employee_update AFT...	0 row(s) affected
✓ 17	13:47:57	UPDATE employees SET salary = 60000 WHER...	1 row(s) affected Rows matched: 1 Cha
✓ 18	13:48:22	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned
✓ 19	13:49:09	SELECT * FROM employee_audit LIMIT 0, 1000	4 row(s) returned
✓ 20	13:50:16	CREATE TRIGGER check_salary BEFORE INSE...	0 row(s) affected
✗ 21	13:51:07	INSERT INTO employees VALUES (107, 'Ravi', 1...	Error Code: 1644. Salary cannot be less

MySQL will reject it and display:

Salary cannot be less than 20000

But, If you run this Query it will succeed as salary is greater than 20000

INSERT INTO employees VALUES (108, 'Meena', 1, 30000);

Query 1 x SQLAddition

Limit to 1000 rows

```
1 • INSERT INTO employees
2 VALUES (108, 'Meena', 1, 30000);
```

Auton
disabl
manu
curre
toge

Context Help

Output

Action Output

#	Time	Action	Message
✓ 17	13:47:57	UPDATE employees SET salary = 60000 WHER...	1 row(s) affected Rows matched: 1 Cha
✓ 18	13:48:22	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned
✓ 19	13:49:09	SELECT * FROM employee_audit LIMIT 0, 1000	4 row(s) returned
✓ 20	13:50:16	CREATE TRIGGER check_salary BEFORE INSE...	0 row(s) affected
✗ 21	13:51:07	INSERT INTO employees VALUES (107, 'Ravi', 1...	Error Code: 1644. Salary cannot be less
✓ 22	13:53:11	INSERT INTO employees VALUES (108, 'Meena'...	1 row(s) affected

Query 1 x

SQL Addition

Limit to 1000 rows

```

1 • SELECT * FROM employees;
2

```

Result Grid

emp_id	emp_name	dept_id	salary
101	Rahul	1	60000.00
108	Meena	1	30000.00
* NULL	NULL	NULL	NULL

employees 8 x

Apply Revert

Context Help

Output

Action Output

#	Time	Action	Message
✓ 18	13:48:22	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned
✓ 19	13:49:09	SELECT * FROM employee_audit LIMIT 0, 1000	4 row(s) returned
✓ 20	13:50:16	CREATE TRIGGER check_salary BEFORE INSE...	0 row(s) affected
✗ 21	13:51:07	INSERT INTO employees VALUES (107, 'Ravi', 1...	Error Code: 1644. Salary cannot be less
✓ 22	13:53:11	INSERT INTO employees VALUES (108, 'Meena'...	1 row(s) affected
✓ 23	13:54:05	SELECT * FROM employees LIMIT 0, 1000	2 row(s) returned

● Fifth Trigger Example--BEFORE UPDATE Trigger:

Suppose the company does not allow salary reduction.

DELIMITER \$\$

```
CREATE TRIGGER prevent_salary_reduction
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < OLD.salary THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Salary reduction is not allowed';

    END IF;
END $$
DELIMITER ;
```

Now, run this Update Query

```
UPDATE employees
SET salary = 40000
WHERE emp_id = 101;
```

If the old salary was ₹50,000, the trigger rejects the update.

But, if you run the below update query, it succeeds as you are not reducing salary

```
UPDATE employees
SET salary = 60000
WHERE emp_id = 101;
```

Here, after salary update

OLD.salary → 50000

NEW.salary → 60000

Query 1

Limit to 1000 rows

```
1 DELIMITER $$
2
3 CREATE TRIGGER prevent_salary_reduction
4 BEFORE UPDATE ON employees
5 FOR EACH ROW
6 BEGIN
7     IF NEW.salary < OLD.salary THEN
8         SIGNAL SQLSTATE '45000'
9         SET MESSAGE_TEXT = 'Salary reduction is not allowed';
10    END IF;
11 END $$
12
13 DELIMITER ;
```

Output

Action Output

#	Time	Action	Message
✓ 20	13:50:16	CREATE TRIGGER check_salary BEFORE INSERT ON employees FOR EACH ROW BEGIN IF NEW.sala...	0 row(s) affected
✗ 21	13:51:07	INSERT INTO employees VALUES (107, 'Ravi', 1, 15000)	Error Code: 1644, Salary cannot be less than
✓ 22	13:53:11	INSERT INTO employees VALUES (108, 'Meena', 1, 30000)	1 row(s) affected
✓ 23	13:54:05	SELECT * FROM employees LIMIT 0, 1000	2 row(s) returned
✓ 24	13:58:46	SELECT * FROM employees LIMIT 0, 1000	2 row(s) returned
✓ 25	13:59:47	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employees FOR EACH ROW BEGIN ...	0 row(s) affected


```
Query 1 x
1 • UPDATE employees
2   SET salary = 70000
3  WHERE emp_id = 101;
```

Output

Action Output

#	Time	Action	Message
✓ 23	13:54:05	SELECT * FROM employees LIMIT 0, 1000	2 row(s) returned
✓ 24	13:58:46	SELECT * FROM employees LIMIT 0, 1000	2 row(s) returned
✓ 25	13:59:47	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employees FOR EACH ROW BEGIN ...	0 row(s) affected
✗ 26	14:00:18	UPDATE employees SET salary = 40000 WHERE emp_id = 101	Error Code: 1644. Salary reduction is not allowed
✗ 27	14:00:21	UPDATE employees SET salary = 40000 WHERE emp_id = 101	Error Code: 1644. Salary reduction is not allowed
✓ 28	14:00:56	UPDATE employees SET salary = 70000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```
Query 1 x
1 • SELECT * FROM employees
2  WHERE emp_id = 101;
```

Result Grid

Filter Rows:

emp_id	emp_name	dept_id	salary
101	Rahul	1	70000.00
NULL	NULL	NULL	NULL

employees 10 x

Output

Action Output

#	Time	Action	Message
✓ 24	13:58:46	SELECT * FROM employees LIMIT 0, 1000	2 row(s) returned
✓ 25	13:59:47	CREATE TRIGGER prevent_salary_reduction BEFORE UPDATE ON employees FOR EACH ROW BEGIN ...	0 row(s) affected
✗ 26	14:00:18	UPDATE employees SET salary = 40000 WHERE emp_id = 101	Error Code: 1644. Salary reduction is not allowed
✗ 27	14:00:21	UPDATE employees SET salary = 40000 WHERE emp_id = 101	Error Code: 1644. Salary reduction is not allowed
✓ 28	14:00:56	UPDATE employees SET salary = 70000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 29	14:01:24	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned

